

Agents et projet final

Un modèle qui agit : des outils Python, une boucle, et ce qui rate avec un petit modèle. Puis les bonnes pratiques avec l'IA, le portfolio GitHub et la présentation finale.

ON REPART AVEC · Un portfolio GitHub complet et sa présentation finale.

01 UN MODÈLE QUI AGIT 02 LA BOUCLE 03 CE QUI RATE
04 LES BONNES PRATIQUES AVEC L'IA 05 TROIS MINUTES DEVANT LES INVITÉS
06 ENTRE DEUX SÉANCES

À LA FIN DE LA SÉANCE

Tu sauras ce qu'est un agent, un modèle, des outils et une boucle, et en lire une trace.
Tu sauras écrire un outil sûr et reconnaître pourquoi un petit agent rate.
Tu sauras présenter ton portfolio en trois minutes à des gens qui ne connaissent pas la data.

Un modèle qui agit

Jusqu'ici, le modèle parlait. Un agent, c'est un modèle à qui l'on donne des outils et le droit de les appeler. Tout ce qui suit sort du notebook, lancé pour de vrai avec son petit modèle.

1. de tête, le vrai modèle

Quelle est la vitesse de Pikachu ?
« **Pikachu se déplace à 100 km/h.** »
vérité : 90, dans le fichier de la séance 2
Combien font $348 * 27$?
« **$348 * 27 = 9164$** »
vérité : 9 396

2. avec des outils : des fonctions Python

```
chercher_pokemon("Pikachu")
```

→ Pikachu : type Electric, 35 PV, attaque 55, défense 40, vitesse 90.

```
chercher_pokemon("Pikachou")
```

→ Aucun Pokémon appelé Pikachou.

```
calculer("348 * 27")
```

→ $348 * 27 = 9396$

```
calculer('__import__("os")..')
```

→ Expression refusée (élément interdit : Call).

```
chercher_dans_mes_notes("à quoi sert la température")
```

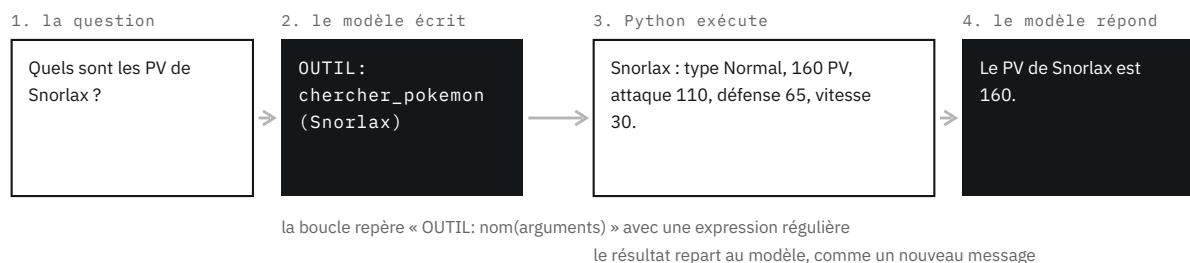
→ Séance 9 : un LLM fait une seule chose, prédire le token suivant, encore et e...

Un outil prend du texte et renvoie du texte ; la calculatrice lit l'expression avec ast et refuse tout ce qui n'est pas un calcul.

Le modèle invente ce qu'il ne sait pas ; un outil va le chercher ou le calcule.

La boucle

La règle est écrite dans le prompt système : pour utiliser un outil, réponds `OUTIL: nom(arguments)` .
La boucle Python repère cette ligne, exécute la fonction et renvoie le résultat au modèle.



Vrai passage du modèle du notebook, température 0 : la seule question du banc de test qu'il a menée au bout.

Un agent, c'est un modèle, des outils et une boucle qui exécute ce qu'il demande.

- DEUX OUTILS À LA SUITE

« Combien fait la vitesse de Pikachu fois 3 ? » demande deux étapes : chercher la vitesse, puis calculer. Lancé pour de vrai, le petit modèle a écrit « `calculer(348 * 27)` », sans « OUTIL: » et avec les nombres de l'exemple du prompt : rien ne s'est exécuté. Le mode démo du notebook, des réponses écrites à la main, montre ce que la boucle doit faire : `chercher_pokemon(Pikachu)` , qui renvoie une vitesse de 90, puis `calculer(90 * 3)` , qui renvoie 270.

Ce qui rate

À 0,5 milliard de paramètres, l'agent rate plus souvent qu'il ne réussit. Le notebook le mesure avec un banc de test.

question	outil attendu	outil demandé
× Quelle est la défense de Charizard ?	chercher_pokemon	aucun
× Combien font $12 * 12 + 1$?	calculer	aucun
× Dans mes notes de cours, c'est quoi un token ?	chercher_dans_mes_notes	aucun
✓ Quels sont les PV de Snorlax ?	chercher_pokemon	chercher_pokemon
× Que disent mes notes de cours sur les trois rôles ?	chercher_dans_mes_notes	aucun
× Combien font $1000 / 8$?	calculer	aucun

Score de l'agent : 1 / 6

vrai passage, Qwen 2.5 0.5B, température 0 : à cette taille, il répond presque toujours de tête

Un banc de test mesure l'agent question par question ; avant de lui confier quoi que ce soit, on le fait passer.

QU'A-T-IL RATÉ ? • SUR PAPIER

Pour chaque question, lis ce que l'agent a fait, puis dis ce qui s'est passé.

Format oublié : il essaie d'appeler un outil, mais sans écrire exactement OUTIL: nom(arguments) : la boucle ne reconnaît pas l'appel

Répond de tête : il n'essaie aucun outil et répond directement, que ce soit juste ou faux

Réussi : il appelle le bon outil au bon format, puis répond à partir du résultat

- « D'après mes notes de cours, quelles sont les étapes du RAG ? » · · réponse : « Chercher_dans_mes_notes(RAG) »
- « Combien fait la vitesse de Pikachu fois 3 ? » · · réponse : « calculer(348 * 27) »
- « Quelle est la défense de Charizard ? » · · réponse : « 150 »
- « Combien font $12 * 12 + 1$? » · · réponse : « 145 »
- « Quels sont les PV de Snorlax ? » · modèle → OUTIL: chercher_pokemon(Snorlax) → outil → Snorlax : type Normal, 160 PV, attaque 110, défense 65, vitesse 30. · réponse : « Le PV de Snorlax est 160. »
- « Quelle est la vitesse de Snorlax ? » · modèle → OUTIL: chercher_pokemon(Snorlax) → outil → Snorlax : type Normal, 160 PV, attaque 110, défense 65, vitesse 30. · réponse : « Snorlax : type Normal, 160 PV, attaque 110, défense 65, vitesse 30. »

Les réponses sont dans le corrigé, en fin de leçon.

Les bonnes pratiques avec l'IA

Savoir comment ça marche change la façon de s'en servir.

Vérifier une date, un chiffre, une citation

Le modèle écrit le plausible, pas le vrai

Ne rien taper qu'on n'écrit pas sur un mur public

Ce qu'on envoie peut être stocké et relu

Faire faire les calculs par Python

Le modèle écrit des chiffres, il ne pose pas l'opération

Se faire expliquer plutôt que copier

Le test : sais-tu le refaire sans lui ?

Garder la main sur envoyer, payer, effacer

Un agent bien construit demande avant les actions importantes

À toi : ton portfolio et ta présentation

Trente minutes pour finir la vitrine de douze séances.

Quatre projets dans ton dépôt GitHub : une analyse, un dashboard et un modèle, Kaggle Titanic, un assistant IA, chacun avec son README.

Le README d'accueil, que la cellule du notebook génère : qui tu es, tes quatre projets, ce que tu as appris.

Ta fiche projet, puis une répétition à deux avec le gabarit des trois minutes.

Trois minutes devant les invités

Les invités ne connaissent ni pandas ni les tokens : montre ce que tu as fait et pourquoi c'est intéressant.

TEMPS

DURÉE CE QUE TU MONTRES

Le projet en une phrase	20 s	« J'ai construit... qui permet de... »
Le problème	30 s	Pourquoi c'était difficile, avec un exemple
La démo	1 min	En vrai : une question à ton assistant, un graphique, ton score
Ce que j'ai appris	40 s	Deux choses nouvelles, et une fois où ça a raté
La suite	30 s	Avec plus de temps, et le lien de ton portfolio

Entre deux séances

Le carnet `12_exercices.ipynb` : 12 exercices, de 🌟 à 🌟🌟🌟, avec vérification automatique, indices et solutions repliées.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Pourquoi la calculatrice du notebook n'utilise-t-elle jamais `eval()` sur ce qu'écrit le modèle ?
 - A. `eval` est trop lent
 - B. `eval` exécuterait n'importe quel code, même un ordre d'effacer des fichiers
 - C. `eval` ne connaît pas les multiplications
 - D. Python interdit `eval`
2. Tu ajoutes un outil `plus_rapide` au dictionnaire `OUTILS`, mais l'agent ne l'appelle jamais. Qu'as-tu oublié ?
 - A. De redémarrer Colab
 - B. De le décrire dans le prompt système : le modèle ne connaît que les outils qu'on lui présente
 - C. De l'écrire en majuscules
 - D. De baisser la température
3. Ton agent peut envoyer des mails. Quelle précaution prendre ?
 - A. Aucune, il sait ce qu'il fait
 - B. Lui faire demander ton accord avant chaque envoi
 - C. Lui donner plus d'outils
 - D. Monter la température
4. Pendant ta présentation de trois minutes, que montres-tu dans la minute de démo ?
 - A. Le code de TF-IDF
 - B. Ton projet en vrai : une question à ton assistant, un graphique ou ton score
 - C. La liste des bibliothèques installées
 - D. Rien, tu racontes

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Un agent, c'est un modèle, des outils et une boucle ; il demande, Python exécute, le résultat repart au modèle.

2. Le modèle ne connaît que les outils qu'on lui décrit, et chaque outil vérifie ce qu'il reçoit.
3. Jamais d'eval sur ce qu'écrit un modèle ; une calculatrice sûre n'accepte que des calculs.
4. Un petit modèle oublie le format ou répond de tête ; un banc de test le mesure avant qu'on lui confie quoi que ce soit.
5. Vérifier, protéger ses données, apprendre plutôt que copier, et garder la main sur les actions importantes.

Corrigé

Qu'a-t-il raté ?

1. **Format oublié** · Le bon outil, presque le bon nom, mais sans « OUTIL: » devant : la boucle ne voit pas d'appel et rend ce texte tel quel comme réponse finale.
2. **Format oublié** · Il tente la calculatrice sans « OUTIL: », donc rien ne s'exécute. Et les arguments sont faux : il recopie l'exemple du prompt au lieu de 90×3 .
3. **Répond de tête** · Aucun outil appelé : il invente un nombre. Le fichier dit 78.
4. **Répond de tête** · C'est juste, mais sans calculatrice : il a répondu de tête et il est tombé juste. Sur un calcul plus long, rien ne garantit la même chance.
5. **Réussi** · OUTIL: chercher_pokemon(Snorlax), exactement le format demandé ; Python lit le fichier, et la réponse reprend les 160 PV.
6. **Réussi** · Le bon outil au bon format ; la réponse recopie toute la fiche au lieu d'une phrase, mais la vitesse, 30, y est bien.

Mini-quiz

1. **B. eval exécuterait n'importe quel code, même un ordre d'effacer des fichiers** · Le modèle, ou quelqu'un qui lui parle, peut écrire du code déguisé en calcul. `calculer()` lit l'expression avec `ast` et n'accepte que des nombres et $+$ $-$ $\times$ $\div$: un appel de fonction est refusé.
2. **B. De le décrire dans le prompt système : le modèle ne connaît que les outils qu'on lui présente** · Le dictionnaire sert à Python pour exécuter l'outil ; le modèle, lui, ne voit que le prompt système. Un outil qu'on ne lui décrit pas n'existe pas pour lui.
3. **B. Lui faire demander ton accord avant chaque envoi** · Même un gros modèle se trompe. Pour les actions qu'on ne peut pas annuler, envoyer, payer, effacer, on garde un humain dans la boucle.
4. **B. Ton projet en vrai : une question à ton assistant, un graphique ou ton score** · Les invités ne connaissent pas les outils : ce qui les convainc, c'est de voir le projet marcher, et même rater une fois, avec ce que tu en as compris.